
1. What is a Class Library?

A **Class Library** is a project type in .NET that contains **reusable code** (classes, interfaces, enums, DTOs, services, helpers, etc.) **without an executable entry point** (Main method).

It compiles into a **DLL (Dynamic Link Library)**, which can be referenced and consumed by:

- ASP.NET Core Web APIs
- MVC / Razor applications
- Console applications
- Other class libraries
- Microservices

Key Characteristics

- Produces a .dll file, not an .exe
 - Cannot run independently
 - Designed for **code reuse and separation of concerns**
 - Commonly used in **Clean Architecture** and **Microservices**
-

2. Why Do We Need a Class Library?

2.1 Separation of Concerns (SoC)

Business logic, domain models, and infrastructure code should not live inside the API/UI project.

2.2 Reusability

The same logic can be reused across multiple applications or services.

2.3 Testability

Class libraries are easier to unit test because they do not depend on UI or hosting concerns.

2.4 Clean Architecture Alignment

Typical Clean Architecture structure:

- **Domain** → Class Library

- **Application** → Class Library
 - **Infrastructure** → Class Library
 - **API/UI** → Executable project
-

3. Real-World Examples of Class Libraries

Library	Purpose
Domain	Entities, Value Objects, Domain Events
Application	DTOs, Interfaces, Use Cases, CQRS
Infrastructure	EF Core, Repositories, External APIs
Shared	Common utilities, constants, helpers
Logging	Custom logging abstractions
Security	JWT, encryption helpers

4. Types of Class Libraries in .NET

4.1 .NET Class Library

- Targets a specific framework (e.g., .NET 8)
- Used when tightly coupled with the application

4.2 .NET Standard Class Library

- Framework-agnostic
 - Can be used by multiple .NET runtimes
 - Less common in modern .NET unless backward compatibility is required
-

5. How to Create a Class Library (Using Visual Studio)

Step 1: Create Project

1. Open **Visual Studio**

2. Click **Create a new project**
3. Select **Class Library**
4. Click **Next**

Step 2: Configure Project

- **Project name:** MyApp.Domain
- **Location:** Same solution folder
- **Framework:** .NET 8.0
- Click **Create**

Visual Studio generates:

```
public class Class1
{
}
```

6. Writing Code in a Class Library

Example: Domain Entity

```
namespace MyApp.Domain.Entities
{
    public class Product
    {
        public int Id { get; private set; }
        public string Name { get; private set; }
        public decimal Price { get; private set; }

        public Product(string name, decimal price)
        {
            Name = name;
        }
    }
}
```

```
        Price = price;
    }
}
}
```

Example: Interface

```
namespace MyApp.Domain.Interfaces
{
    public interface IProductRepository
    {
        Task<IEnumerable<Product>> GetAllAsync();
        Task<Product?> GetByIdAsync(int id);
        Task AddAsync(Product product);
    }
}
```

7. Referencing a Class Library in Another Project

Step 1: Add Project Reference

1. Right-click **API project**
2. Click **Add → Project Reference**
3. Select the class library
4. Click **OK**

Step 2: Use the Library

```
using MyApp.Domain.Entities;
```

8. Class Library in Clean Architecture (Important)

Correct Placement

Layer	Class Library?
Domain	Yes
Application	Yes
Infrastructure	Yes
API	No (Executable)

Dependency Rule

- API → Application
- Application → Domain
- Infrastructure → Application & Domain
- Domain → **No dependency on others**

9. Common Mistakes

Mistake	Why It's Wrong
Adding EF Core in Domain	Domain must be persistence-agnostic
Adding Controllers in class library	Controllers belong to API
Business logic in API	Violates SoC
Circular project references	Breaks Clean Architecture

10. When Should You Create a Class Library?

Create a class library when:

- Code needs to be reused
 - Logic is independent of UI or hosting
 - You are following Clean Architecture or DDD
 - You want better maintainability and testability
-

11. Simple Solution Structure Example

MyApp.sln

|

└─ MyApp.API (ASP.NET Core Web API)

└─ MyApp.Application (Class Library)

└─ MyApp.Domain (Class Library)

└─ MyApp.Infrastructure (Class Library)

12. Summary

- A **Class Library** is a reusable, non-executable project
- Compiles into a **DLL**
- Core building block of **Clean Architecture**
- Improves **maintainability, testability, and scalability**
- Widely used in enterprise-grade .NET applications